جزوه درجه یک سیستم عامل

فصل ۴ - بخش ۱: کلیات نخ (Thread)

در فصل ۳ یاد گرفتیم که "پردازه" (Process) یک برنامه در حال اجرا است. حالا می‌خواهیم این مفهوم را خُردتر کنیم.

نخ (Thread) چیست؟ یک "نخ" (یا "رشته")، **واحد اساسی استفاده از CPU** است.

بیایید پردازه و نخ را با یک مثال مقایسه کنیم:

- **پردازه (Process):** مثل یک "کارگاه" کامل است. این کارگاه شامل **منابع** زیادی است: کد برنامه (دستورالعمل‌ها)، داده‌ها (مواد اولیه)، فایل‌های باز (ابزارها) و...
- **نخ (Thread):** مثل یک "کارگر" *درون* آن کارگاه است.

یک پردازه تک-نخی (مدل فصل ۳) کارگاهی است که فقط **یک کارگر** دارد. یک پردازه چند-نخی (مدل این فصل) کارگاهی است که **چندین کارگر** دارد که *همزمان* با هم کار می‌کنند.

نخ‌ها چه چیزی را به اشتراک می‌گذارند؟ تمام کارگرها (نخ‌ها) در یک کارگاه (پردازه)، در منابع اصلی **مشترک** هستند:

- **بخش کد (Code):** همه از یک کتابچه دستورالعمل استفاده می‌کنند.
- **بخش داده (Data):** همه به یک انبار مواد اولیه دسترسی دارند.
- **فایل‌های باز:** همه از ابزارهای مشترک کارگاه استفاده می‌کنند.

نخ‌ها چه چیزی را به صورت اختصاصی دارند؟ هر کارگر (نخ) برای اینکه بتواند کار خودش را مستقل انجام دهد، به چند چیز *اختصاصی* نیاز دارد:

1. **شناسه نخ (Thread ID):** نام کارگر.
2. **شمارنده برنامه (Program Counter - PC):** هر کارگر می‌داند خودش در کجای کتابچه دستورالعمل است.
3. **مجموعه ثبات‌ها (Registers):** ابزارهای دم‌دستی و شخصی هر کارگر.
4. **پشته (Stack):** دفترچه یادداشت شخصی هر کارگر (برای متغیرهای محلی توابع).

۴-۱-۱: انگیزه (چرا به نخ نیاز داریم؟)

امروزه تقریباً تمام برنامه‌های مدرن چندنخی هستند:

- **مرورگر وب:** یک نخ متن و عکس‌ها را نشان می‌دهد، در حالی که یک نخ دیگر در پس‌زمینه در حال دانلود کردن ادامه صفحه است.
- **واژه‌پرداز:** یک نخ ورودی شما را تایپ می‌کند، یک نخ دیگر در پس‌زمینه غلط املایی را چک می‌کند، و نخ سوم گرافیک‌ها را نمایش می‌دهد.
- **برنامه ادیت عکس:** وقتی ۱۰۰ عکس را باز می‌کنید، برنامه ۱۰۰ نخ ایجاد می‌کند که هر کدام *همزمان* در حال ساختن یک عکس کوچک (Thumbnail) هستند.

مثال کلیدی: سرور وب (سوال امتحانی) تصور کنید یک سرور وب (مثل سرور گوگل) دارید:

- **مدل قدیمی (تک-نخی):** سرور در هر لحظه فقط به یک نفر می‌تواند پاسخ دهد. بقیه باید در صف بمانند. (فاجعه!)
- **مدل بهتر (چند-پردازه‌ای):** سرور به ازای *هر درخواست* یک **پردازه** جدید `fork` می‌کند (کاری که در فصل ۳ یاد گرفتیم).
  - **مشکل:** `fork` کردن (ایجاد پردازه) بسیار **سنگین، کند** و **پرهزینه** است (چون باید یک کارگاه جدید از صفر بسازیم).
- **مدل مدرن (چند-نخی):**
  1. سرور (یک پردازه) با یک نخ اصلی به درخواست‌ها "گوش" می‌دهد.
  2. به محض رسیدن یک درخواست جدید، به جای ساختن یک *پردازه* جدید، یک **نخ** جدید (یک کارگر جدید) ایجاد می‌کند تا به آن درخواست رسیدگی کند.
  3. نخ اصلی بلافاصله به "گوش دادن" برای درخواست‌های بعدی برمی‌گردد.
  - **نتیجه:** ساختن "نخ" (کارگر) **هزاران بار سریع‌تر** از ساختن "پردازه" (کارگاه) است.

۴-۱-۲: منافع (مزایای ۴ گانه نخ‌ها)

1. **پاسخگویی (Responsiveness):**
   - مهم‌ترین مزیت برای کاربر.
   - اگر یک نخ (مثلاً نخ محاسبات سنگین) مسدود (Block) شود، نخ‌های دیگر (مثل نخ رابط کاربری) به کار خود ادامه می‌دهند.
   - **مثال:** در یک برنامه تک-نخی، اگر دکمه‌ای را بزنید که یک عملیات سنگین را شروع کند، کل برنامه "قفل" می‌کند. در برنامه چند-نخی، رابط کاربری روان باقی می‌ماند.
1. **اشتراک منابع (Resource Sharing):**
   - نخ‌ها *به طور پیش‌فرض* حافظه و منابع پردازه والد خود را به اشتراک می‌گذارند.
   - (برخلاف پردازه‌ها که برای اشتراک حافظه به تکنیک‌های پیچیده IPC مثل "حافظه مشترک" یا "عبور پیغام" نیاز داشتند).
1. **صرفه اقتصادی (Economy):**
   - **ساختن** یک نخ بسیار ارزان‌تر و سریع‌تر از ساختن یک پردازه است.
   - **جابجایی (Context Switch)** بین دو نخ (در یک پردازه) بسیار سریع‌تر از جابجایی بین دو پردازه است (چون نیازی به تغییر نقشه حافظه نیست).
1. **توسعه‌پذیری (Scalability):**
   - در دنیای امروز که CPUها **چند هسته‌ای (Multi-core)** هستند، این مزیت می‌درخشد.
   - یک پردازه تک-نخی، حتی اگر شما ۱۶ هسته CPU هم داشته باشید، فقط می‌تواند روی **یک** هسته اجرا شود.
   - اما یک پردازه چند-نخی می‌تواند نخ‌های خود را *همزمان* روی تمام هسته‌ها پخش کند و کار را به صورت **موازی** انجام دهد.

فصل ۴ - بخش ۲: برنامه‌نویسی چند هسته‌ای (Multi-core)

امروزه دیگر سرعت CPUها (ساعت پالس) زیاد نمی‌شود، بلکه **تعداد هسته‌ها (Cores)** زیاد می‌شود. برنامه‌نویسی چندنخی تنها راه استفاده کامل از این سخت‌افزارهاست.

الف) همروندی (Concurrency) در مقابل موازی‌سازی (Parallelism)

این دو مفهوم مهم‌ترین بخش این فصل هستند و **نباید** با هم اشتباه گرفته شوند.

۱. همروندی (Concurrency):

- **کجا؟** روی یک سیستم **تک-هسته‌ای (Single-core)**.
- **چطور؟** سیستم‌عامل با "زرنگی" بین نخ‌ها *جابجا (Switch)* می‌شود. `1T` کمی اجرا می‌شود، بعد `2T`، بعد `3T`، و دوباره `1T`...
- **نتیجه:** هر سه نخ در حال "پیشرفت" هستند و به نظر می‌رسد همزمان اجرا می‌شوند، اما در هر لحظه **فقط یکی** از آن‌ها واقعاً در حال اجراست.
- **تشبیه:** یک آشپز (تک هسته) که همزمان ۳ غذا (۳ نخ) را می‌پزد. اولی را هم می‌زند، بعد سراغ دومی می‌رود، بعد سومی را چک می‌کند. هر ۳ غذا در حال پختن هستند (همروندی)، اما آشپز در هر لحظه فقط روی یکی کار می‌کند.

۲. موازی‌سازی (Parallelism):

- **کجا؟** فقط روی یک سیستم **چند-هسته‌ای (Multi-core)**.
- **چطور؟** `1T` روی "هسته ۱"، `2T` روی "هسته ۲" و `3T` روی "هسته ۳" **دقیقاً در یک لحظه** اجرا می‌شوند.
- **نتیجه:** کارها واقعاً به صورت همزمان انجام می‌شوند.
- **تشبیه:** ۳ آشپز (چند هسته) که هر کدام همزمان روی یک غذا (یک نخ) کار می‌کنند.
- **نکته کلیدی:** شما می‌توانید همروندی را ***بدون*** موازی‌سازی داشته باشید (روی تک هسته)، اما موازی‌سازی *نوعی* از همروندی است.

ب) قانون اَمدال (Amdahl's Law)

این قانون یک زنگ خطر مهم است. می‌گوید اضافه کردن هسته CPU همیشه به معنی سریع‌تر شدن برنامه نیست.

- هر برنامه‌ای یک بخش **سریال (Serial)** دارد (بخش‌هایی که *نمی‌توان* آن‌ها را موازی کرد، مثلاً `S`) و یک بخش **موازی (Parallel)** (مثلاً `1-S`).
- **قانون امدال می‌گوید:** "سرعت نهایی شما همیشه توسط آن بخش سریال محدود می‌شود."
- **مثال:** اگر ۵۰٪ برنامه شما سریال باشد (`S=0.5`)، حتی اگر *بی‌نهایت* هسته CPU هم داشته باشید، برنامه شما حداکثر **۲ برابر** (`S/1`) سریع‌تر می‌شود. افزودن هسته بیشتر، دیگر کمکی نمی‌کند.

ج) چالش‌های برنامه‌نویسی چند هسته‌ای

برنامه‌نویسی موازی بسیار سخت است و ۵ چالش اصلی دارد:

1. **شناسایی وظیفه‌ها:** پیدا کردن بخش‌هایی از کد که واقعاً مستقل هستند و می‌توانند موازی شوند.
2. **تعادل (Balance):** کارها باید مساوی تقسیم شوند. اگر یک هسته ۹۰٪ کار را انجام دهد و هسته دیگر ۱۰٪، ما فقط ۱۰٪ بهره برده‌ایم.
3. **شکستن داده‌ها:** داده‌ها چطور باید بین هسته‌ها تقسیم شوند؟
4. **وابستگی داده‌ها:** اگر نخ ۲ به نتیجه کار نخ ۱ نیاز داشته باشد، چه؟ (این ما را به بحث "همگام‌سازی" در فصل‌های بعد می‌رساند).
5. **آزمون و عیب‌یابی:** عیب‌یابی برنامه‌های موازی یک کابوس است، چون ترتیب اجرا ممکن است هر بار عوض شود و خطاها به صورت تصادفی رخ دهند.

د) انواع موازی‌سازی

دو راهبرد اصلی برای موازی کردن کارها وجود دارد:

۱. موازی‌سازی داده (Data Parallelism):

- **ایده:** *عملیات* یکسان، *داده‌های* متفاوت.
- **چطور؟** ما یک مجموعه داده بزرگ (مثلاً یک آرایه) را به چند بخش تقسیم می‌کنیم و *همان* عملیات را روی هر بخش اجرا می‌کنیم.
- **مثال:** برای جمع کردن یک آرایه ۱۰۰۰تایی روی ۴ هسته:
  - هسته ۱: خانه‌های ۰ تا ۲۴۹ را جمع می‌زند.
  - هسته ۲: خانه‌های ۲۵۰ تا ۴۹۹ را جمع می‌زند.
  - ... (همه یک کار، یعنی "جمع زدن" را انجام می‌دهند).

۲. موازی‌سازی وظیفه (Task Parallelism):

- **ایده:** *عملیات* متفاوت، *داده‌های* یکسان یا متفاوت.
- **چطور؟** ما وظایف (کارهای) مختلف را به هسته‌های مختلف می‌سپاریم.
- **مثال:** پردازش یک آرایه روی ۴ هسته:
  - هسته ۱: "میانگین" آرایه را حساب می‌کند.
  - هسته ۲: "بزرگترین" عدد آرایه را پیدا می‌کند.
  - هسته ۳: "کوچکترین" عدد آرایه را پیدا می‌کند.
  - ... (هر هسته کار متفاوتی انجام می‌دهد).

این دو روش می‌توانند با هم ترکیب شوند.